

Text Splitters in LangChain

1. What is Text Splitting?

- **Definition:**
Splitting a large text (PDF, article, book, HTML, code) into **smaller chunks** that an **LLM can handle effectively**.
- **Why needed?**
 - LLMs have **context length limits** (e.g., 50k tokens).
 - Large docs (books, reports) exceed this → cannot be directly processed.
 - Splitting ensures:
 - Better embeddings
 - More accurate semantic search
 - Higher-quality summarization
- **Tool:**
LangChain provides **TextSplitter classes** to automate this process.

2. Why is Text Splitting Important?

- ✓ **Reason 1: Overcome LLM limitations**
 - Models can't take unlimited input.
 - Example: PDF with 1,00,000 words > 50k token limit → needs splitting.
- ✓ **Reason 2: Improve downstream tasks**
 - **Embeddings:** Smaller chunks capture meaning better.
 - **Semantic search:** Queries map more accurately to smaller sections.
 - **Summarization:** Prevents hallucination & drifting off-topic.
- ✓ **Reason 3: Efficient resource usage**
 - Smaller chunks = less memory usage.
 - Parallel processing possible → faster.

3. Four Major Types of Text Splitters

- ◆ **3.1 Length-based Splitters**
 - Simplest and fastest method.
 - Define **chunk size** (by characters or tokens).
 - Example: Every 100 characters → new chunk.

Code Example:

```
from langchain.text_splitter import CharacterTextSplitter

text = "Your long document text here..."
splitter = CharacterTextSplitter(chunk_size=100, chunk_overlap=0)
chunks = splitter.split_text(text)
print(chunks[:2]) # First two chunks
```

- **Pros:**
 - Easy, fast.
- **Cons:**
 - Ignores linguistic structure.
 - Can cut words/sentences midway.

◆ 3.2 Chunk Overlap

- Adds overlap between chunks to **preserve context**.
- Example: chunk_size=100, overlap=20 → last 20 chars of one chunk repeat in the next.

Best Practice:

- Set overlap = 10–20% of chunk size.
- **Pros:** Retains continuity.
- **Cons:** More computation (extra chunks).

◆ 3.3 Structure-based Splitters

- Respect natural text structure → paragraphs → sentences → words → characters.
- Most common: **RecursiveCharacterTextSplitter**.
- Tries largest unit first (paragraph), if too long → breaks down further.

Code Example:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(chunk_size=200, chunk_overlap=20)
chunks = splitter.split_text(text)
```

- **Pros:** Preserves meaning better than character splitter.
- **Cons:** Slightly slower.

👉 **Most widely used splitter in production.**

◆ 3.4 Document-based Splitters

- For **non-plain text docs** (code, markdown, HTML).
- Uses **language-specific separators** (e.g., class, def, {}, <h1>).

Example (Python code splitting):

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

python_code = """class MyClass: ..."""
splitter = RecursiveCharacterTextSplitter.from_language(
    language="python", chunk_size=200, chunk_overlap=20
)
chunks = splitter.split_text(python_code)
```

- **Use cases:**
 - Codebases
 - Markdown docs
 - HTML/XML

◆ 3.5 Semantic Meaning-based Splitters (Experimental)

- Uses **embeddings + similarity** to decide split points.
- Idea: If two sentences are semantically very different → split there.
- Example: Agriculture + IPL in same paragraph → split them separately.

Code Example:

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()
splitter = SemanticChunker(embeddings)
chunks = splitter.split_text(text)
```

- **Pros:** Preserves meaning perfectly.
- **Cons:** Still experimental, inconsistent results, computationally expensive.
- **Future potential:** Likely to become standard as embedding models improve.

4. Key Parameters

- **chunk_size:** Max size of each chunk (characters/tokens).
- **chunk_overlap:** Context carried between chunks (usually 10–20%).

- **separators:** Used in recursive splitting (`\n\n` for paragraphs, `.` for sentences).
-

5. Best Practices (Interview Gold)

- Always **use overlap** to avoid losing context.
 - Choose chunk size depending on task:
 - **Embeddings/Semantic Search:** Smaller chunks (200–500 tokens).
 - **Summarization/QA:** Larger chunks (500–1000 tokens).
 - For **code/structured docs**, use document-based splitters.
 - For **domain-specific tasks**, experiment with semantic splitters.
-

6. Interview Q&A

Q1. Why not feed entire text to LLM?

- Context length limit.
- Large inputs reduce quality → hallucination, inefficiency.

Q2. Difference between `CharacterTextSplitter` and `RecursiveCharacterTextSplitter`?

- `Character` → splits blindly by size.
- `Recursive` → respects structure (paragraphs, sentences, words).

Q3. When to use chunk overlap?

- When continuity between chunks is needed (embeddings, search).

Q4. What are semantic splitters?

- Splitting based on **meaning similarity** instead of raw size/structure.
- Experimental, but promising.

Q5. Which splitter is most common in production?

- **`RecursiveCharacterTextSplitter`** (best balance between accuracy and speed).
-

7. Real-World Applications

- **Chat with PDFs/books:** Break long docs into chunks for QA bots.
 - **Code assistants:** Split codebase by functions/classes.
 - **Knowledge management:** Split Markdown docs, HTML pages.
 - **Enterprise search:** Ingest reports, policies, FAQs.
 - **Summarization pipelines:** Split long reports into manageable parts.
-

✅ Summary:

- Use **length-based** for speed.
 - Use **recursive** for better quality (default in most RAG apps).
 - Use **document-based** for structured data (code, markdown).
 - Explore **semantic splitters** for advanced meaning-based segmentation.
-